microgpt

Lecture Notes: the complete algorithm of a GPT in 200 lines

Course notes built around Andrej Karpathy's *microgpt* (12 February 2026)
with references to the surrounding literature

Compiled October 11, 2026

Abstract

These notes walk through *microgpt* (Karpathy, 2026b,d), a single 200-line, dependency-free Python file that trains and samples from a GPT-style language model. The file contains everything that is algorithmically essential: a dataset of documents, a tokenizer, a scalar autograd engine, a GPT-2-like Transformer, the Adam optimizer, a training loop and an inference loop. Karpathy's thesis is that everything else in a production system is "just efficiency". We follow the structure of the original post section by section, but slow down to derive the mathematics, to connect each design decision to the papers that introduced it, and to say precisely what changes on the way to a model such as ChatGPT. A complete listing of the program appears in section A; exercises are collected in section 13.

Contents

1 Orientation **3**
- 1.1 What microgpt is . . . 3
- 1.2 The seven components . . . 3
- 1.3 Where to get the code . . . 3
- 1.4 Notation . . . 3

2 Dataset: text as a stream of documents **4**
- 2.1 The code . . . 4
- 2.2 Why "documents" . . . 4
- 2.3 Literature . . . 4

3 Tokenizer: characters as discrete symbols **4**
- 3.1 The code . . . 4
- 3.2 Tokens are symbols, not numbers . . . 5
- 3.3 The role of `BOS` . . . 5
- 3.4 Literature: from characters to byte-pair encoding . . . 5

4 Autograd: the chain rule as a data structure **5**
- 4.1 The `Value` class . . . 5
- 4.2 The backward pass . . . 6
- 4.3 A worked example . . . 7
- 4.4 What PyTorch does differently . . . 7
- 4.5 Literature . . . 8

5 Parameters: the knowledge of the model **8**

5.1 Counting the parameters . . . 9
5.2 Why these initialisation choices . . . 9

6 Architecture: a GPT-2-style Transformer, one token at a time 9
6.1 Three helpers . . . 9
6.2 The model function . . . 10
6.3 Embeddings . . . 11
6.4 Attention . . . 11
6.5 The KV cache, during training . . . 13
6.6 The MLP block . . . 14
6.7 Residual connections and pre-normalisation . . . 14
6.8 Output . . . 14
6.9 Cost of one token . . . 14

7 Training loop: loss, gradients and Adam 14
7.1 Next-token prediction . . . 15
7.2 Cross-entropy loss . . . 15
7.3 Batch size one, and what it means for the loss curve . . . 16
7.4 Backward . . . 16
7.5 Adam . . . 17

8 Inference: sampling new names 17
8.1 Temperature . . . 18
8.2 What the samples show . . . 18

9 Running it 18

10 Progression: building the program in six steps 19

11 From microgpt to a production LLM 20
11.1 Data . . . 20
11.2 Tokenizer . . . 20
11.3 Autograd and hardware . . . 21
11.4 Architecture . . . 21
11.5 Training and optimisation . . . 21
11.6 Post-training . . . 21
11.7 Inference . . . 21

12 Frequently asked questions 22

13 Exercises 22

A Complete source of `microgpt.py` 24

1 Orientation

1.1 What microgpt is

Karpathy describes microgpt as an "art project": the culmination of micrograd (Karpathy, 2020), makemore (Karpathy, 2022a), nanoGPT (Karpathy, 2022b) and llm.c (Karpathy, 2024b), each of which tried to reduce a large language model (LLM) to fewer moving parts. The program imports only `os`, `math` and `random`; it downloads a text file of about 32,000 names, trains a tiny Transformer on them for 1,000 steps, and then prints twenty invented names such as *kamon*, *karai* and *alerin*.

> "This file contains the full algorithmic content of what is needed: dataset of documents, tokenizer, autograd engine, a GPT-2-like neural network architecture, the Adam optimizer, training loop, and inference loop. Everything else is just efficiency. I cannot simplify this any further." — Karpathy (2026b)

The claim deserves to be taken seriously. Every component of a frontier model has a counterpart in these 200 lines, and the counterparts differ in scale, parallelism and engineering polish rather than in kind. These notes therefore have two goals:

1. to make sure the reader can explain every line of the program, and
2. to make sure the reader knows which paper, idea or engineering constraint each line descends from.

1.2 The seven components

Figure 1 shows the pipeline. The numbering matches the sections of these notes.

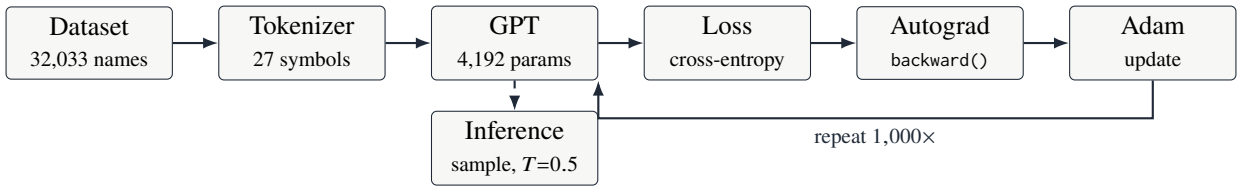


Figure 1: The microgpt pipeline. Training repeats the forward–loss–backward–update loop once per document. Inference reuses the same model function, feeding each sampled token back in.

1.3 Where to get the code

- Blog post (the primary text for these notes): https://karpathy.github.io/2026/02/12/microgpt/
- The gist `microgpt.py`: https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95
- The step-by-step build `build_microgpt.py`, whose revision history is `train0.py` through `train5.py`: https://gist.github.com/karpathy/561ac2de12a47cc06a23691e1be9543a
- A Google Colab notebook (Karpathy, 2026c), and a one-page web rendering at https://karpathy.ai/microgpt.html.

Run it with `python microgpt.py`. There is nothing to install. On a laptop the run takes a minute or two; the timing we measured for these notes is reported in section 9.

1.4 Notation

We write V for the vocabulary size ($V = 27$), d for the embedding width ($d = 16$, called `n_embd` in the code), H for the number of heads ($H = 4$), $d_h = d/H$ for the head width ($d_h = 4$), L for the number of layers ($L = 1$)

and $T_{\max}$ for the context length ($T_{\max} = 16$, `block_size`). Vectors are columns in $\mathbb{R}^d$ unless stated otherwise; a weight matrix $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ acts by Wx. This matches the code, where `matrix(nout, nin)` builds a list of `nout` rows each of length `nin`, and `linear(x, w)` computes one dot product per row.

2 Dataset: text as a stream of documents

2.1 The code

```
# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")
```

The file `names.txt` comes from the makemore project (Karpathy, 2022a). It contains 32,033 lowercase first names, one per line (*emma, olivia, ava, isabella,* . . .). Each line is treated as one *document*. The longest name has 15 characters, which is why the context length is set to 16 later on.

2.2 Why "documents"

The language-modelling task is to learn the probability distribution of sequences in the training data and then to sample new sequences from it. In production the documents are web pages, books and code (Gao et al., 2020; Penedo et al., 2024; Raffel et al., 2020); here they are names. Nothing else in the program depends on what the documents contain, which is the point of the FAQ entry "What if I change the dataset?": swap in city names or short poems and the remaining 190 lines are unchanged.

Key idea: a conversation is just a document

From the model's point of view, a ChatGPT conversation is a single, oddly formatted document. The system prompt and the user's message are the prefix; the model's reply is a statistical *completion* of that document, produced one token at a time. microgpt completing *k-a-r* into *karai* and ChatGPT completing a prompt into a paragraph are the same operation.

2.3 Literature

Treating text as a sequence to be predicted goes back to Shannon (1948, 1951), who measured how predictable English letters are and introduced the entropy of a source, which we meet again when we interpret the loss (section 7.2). Neural language models in the modern sense begin with Bengio et al. (2003); character-level neural generation of text, the closest ancestor of microgpt's task, was popularised by Sutskever et al. (2011), Graves (2013) and Karpathy's own essay on recurrent networks (Karpathy, 2015).

3 Tokenizer: characters as discrete symbols

3.1 The code

```
# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
```

```
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")
```

The unique characters in the names are the 26 lowercase letters, sorted so that a$\mapsto 0, \ldots,$ z$\mapsto 25$. One extra symbol, `BOS` ("beginning of sequence"), gets id 26, so $V = 27$. Encoding is `uchars.index(ch)`; decoding is `uchars[token_id]`.

3.2 Tokens are symbols, not numbers

The integer ids carry no arithmetic meaning: 4 is not "twice" 2, and `e` is not between `d` and `f` in any sense the model is told about. Karpathy's phrasing is that the ids "might as well be different emoji". The first thing the network does with an id is to look up a learned vector for it (section 6.3), which is where meaning is eventually attached.

3.3 The role of `BOS`

Every training document is wrapped as `[BOS, e, m, m, a, BOS]`. The leading `BOS` gives the model a starting context from which to predict the first letter; the trailing `BOS` is a target the model must learn to emit when a name is finished. At inference the loop starts by feeding `BOS` and stops when `BOS` is sampled. A single symbol therefore plays the roles that other systems split into `<s>`/`</s>` or `<|endoftext|>`.

3.4 Literature: from characters to byte-pair encoding

Production tokenizers do not work on single characters. GPT-2 (Radford et al., 2019) and tiktoken (OpenAI, 2022) use byte-level *byte-pair encoding* (BPE). BPE was invented as a compression algorithm by Gage (1994) and brought to neural machine translation by Sennrich et al. (2016): starting from bytes, repeatedly merge the most frequent adjacent pair into a new symbol until the vocabulary reaches a target size (around 50k for GPT-2, around 100k for GPT-4). Common words become one token and rare words are split into pieces, so the model sees more content per position and can attend over longer stretches of text. SentencePiece (Kudo and Richardson, 2018) is the other widely used implementation. Karpathy's tokenizer lecture (Karpathy, 2024a) builds a BPE tokenizer from scratch and is the natural next step after these notes.

4 Autograd: the chain rule as a data structure

Training adjusts parameters to lower a loss, so for every parameter θ we need $\partial \mathcal{L}/\partial\theta$: "if I nudge this number up a little, does the loss go up or down, and by how much?" The computation has thousands of inputs and one scalar output, which is exactly the setting in which *reverse-mode* automatic differentiation, better known as backpropagation, is efficient: one backward sweep yields all gradients at a cost comparable to one forward pass.

4.1 The `Value` class

```
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads')

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forward pass
        self.grad = 0                   # derivative of the loss w.r.t. this node, calculated in backward pass
```

```
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1
```

A `Value` wraps one floating-point number and records how it was produced: which nodes it was computed from (`_children`) and the derivative of the output with respect to each of them (`_local_grads`). Every arithmetic operation returns a fresh node, so a forward computation silently builds a directed acyclic graph (DAG) whose leaves are parameters and constants and whose single root is the loss.

Only six primitive operations are needed; everything else is defined in terms of them (subtraction is addition of a negation, division is multiplication by a power of -1).

Operation	Forward	Local gradients stored
`a + b`	$a + b$	$\partial/\partial a = 1, \quad \partial/\partial b = 1$
`a * b`	$a \cdot b$	$\partial/\partial a = b, \quad \partial/\partial b = a$
`a ** n`	a^n	$\partial/\partial a = n\,a^{n-1}$
`a.log()`	$\ln a$	$\partial/\partial a = 1/a$
`a.exp()`	e^a	$\partial/\partial a = e^a$
`a.relu()`	$\max(0, a)$	$\partial/\partial a = \mathbf{1}[a > 0]$

Table 1: The six "lego blocks" of the autograd engine. Each stores its local derivative at construction time, using the forward values that are available at that moment.

Remark 4.1. The local gradient is computed and stored during the *forward* pass, when the operand values are at hand. The backward pass therefore never recomputes anything; it only multiplies and adds stored numbers. This is the design of micrograd (Karpathy, 2020) and, conceptually, of PyTorch's autograd (Paszke et al., 2017).

4.2 The backward pass

```
    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
```

```
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad
```

Three things happen.

Topological sort. A depth-first search from the root appends each node to `topo` *after* all of its children, so `topo` lists nodes from leaves to root. Reversing it gives an order in which every node is visited before anything it depends on; that is exactly what the chain rule needs, because a node's gradient must be final before it is propagated to its children.

Seed. The loss node receives $\partial \mathcal{L}/\partial \mathcal{L} = 1$.

Chain rule with accumulation. For a node v with child c and stored local gradient $\partial v/\partial c$,

$$\frac{\partial \mathcal{L}}{\partial c} \mathrel{+}= \frac{\partial v}{\partial c} \cdot \frac{\partial \mathcal{L}}{\partial v}. \tag{1}$$

The `+=` is essential. If c feeds into several nodes, the loss depends on c along several paths, and the multivariable chain rule says the total derivative is the *sum* over paths. Karpathy's analogy: a car twice as fast as a bicycle, which is four times as fast as a walker, is $2 \times 4 = 8$ times as fast as the walker; rates multiply along a path, and when there are several paths, their contributions add.

4.3 A worked example

```
a = Value(2.0)
b = Value(3.0)
c = a * b       # c = 6.0
L = c + a       # L = 8.0
L.backward()
print(a.grad)   # 4.0 (dL/da = b + 1 = 3 + 1, via both paths)
print(b.grad)   # 2.0 (dL/db = a = 2)
```

Interpretation: $a.\text{grad} = 4$ means that increasing a by 0.001 raises L by about 0.004. The sign gives the direction of influence, the magnitude its steepness. Gradient descent uses exactly this information in reverse: to lower L, move each input a little against its gradient.

4.4 What PyTorch does differently

The same program in PyTorch gives identical numbers:

```
import torch
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
c = a * b
L = c + a
```

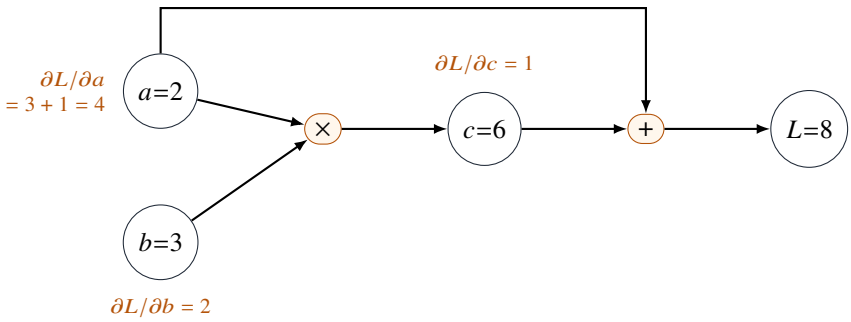


Figure 2: The graph for $L = ab + a$. The node a has two parents ($\times$ and $+$), so its gradient accumulates from both: $b \cdot 1 + 1 \cdot 1 = 4$.

```
L.backward()
print(a.grad)   # tensor(4.)
print(b.grad)   # tensor(2.)
```

PyTorch's engine (Paszke et al., 2019) runs the same algorithm over *tensors*, so that one node of its graph can stand for millions of scalars and one backward operation can be a fused GPU kernel. microgpt's graph contains one node per scalar operation. Running one training step on the name "emma" creates on the order of a hundred thousand `Value` objects (we count them in section 9), which is why the pure-Python version is slow and why nobody trains real models this way. The mathematics, however, is identical.

4.5 Literature

Reverse-mode automatic differentiation was published by Linnainmaa (1976); its application to neural networks under the name backpropagation is due to Rumelhart et al. (1986). Griewank and Walther (2008) is the standard mathematical treatment, and Baydin et al. (2018) survey the field for a machine-learning audience, including the distinction between forward and reverse mode and the role of the computational graph. Karpathy's 2.5-hour micrograd lecture (Karpathy, 2022c) builds the `Value` class line by line.

5 Parameters: the knowledge of the model

```
n_layer = 1     # depth of the transformer neural network (number of layers)
n_embd = 16     # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4      # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(
    ↪ vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row]
print(f"num params: {len(params)}")
```

Every parameter is a `Value` initialised from $\mathcal{N}(0,\ 0.08^2)$. The dictionary `state_dict` borrows PyTorch's name and organises the parameters into matrices; `params` flattens them into one list so that the optimizer can loop over them.

5.1 Counting the parameters

Name	Shape	Formula	Count
`wte`	$V \times d$	27×16	432
`wpe`	$T_{\max} \times d$	16×16	256
`lm_head`	$V \times d$	27×16	432
`attn_wq, attn_wk, attn_wv, attn_wo`	$4 \times (d \times d)$	$4 \times 16 \times 16$	1,024
`mlp_fc1`	$4d \times d$	64×16	1,024
`mlp_fc2`	$d \times 4d$	16×64	1,024
Total			**4,192**

Table 2: Parameter inventory for the default configuration ($L = 1$). Per layer, the Transformer block holds $12d^2 = 3{,}072$ parameters, split evenly between attention ($4d^2$) and the MLP ($8d^2$).

For comparison: GPT-2 (Radford et al., 2019) has 1.5 billion parameters in its largest configuration (Karpathy quotes 1.6B), GPT-3 has 175 billion (Brown et al., 2020), and current frontier models have hundreds of billions to trillions. The $12d^2$ per-layer rule generalises: for $d = 1600$ and 48 layers, the GPT-2 XL block parameters come to $48 \times 12 \times 1600^2 \approx 1.47$B.

5.2 Why these initialisation choices

- **Small Gaussian.** Weights must be random so that different neurons compute different things (symmetry breaking), and small so that early activations and softmaxes are not saturated. The scale 0.08 is close to $1/\sqrt{d} = 0.25$ divided by about three; the classical analysis of why the scale should shrink with fan-in is Glorot and Bengio (2010). GPT-2 used 0.02 with $d \geq 768$.
- **No biases.** Modern LLMs (LLaMA and successors, Touvron et al. 2023) drop bias vectors; they add parameters without measurable benefit once normalisation layers are present.
- **No weight tying.** GPT-2 shares `wte` with `lm_head` (Press and Wolf, 2017); microgpt keeps them separate, which is simpler to read and costs only 432 extra parameters here.

6 Architecture: a GPT-2-style Transformer, one token at a time

The model is a *stateless function*: given a token id, its position, the parameters, and the keys and values cached from earlier positions, it returns V logits, one score per possible next token. Karpathy follows GPT-2 (Radford et al., 2019) with three simplifications, announced in a comment in the source:

$$\textit{layernorm} \to \textit{rmsnorm}, \qquad \textit{no biases}, \qquad \textit{GeLU} \to \textit{ReLU}.$$

6.1 Three helpers

```
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
```

```
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

linear is a matrix–vector product $y = Wx$, one dot product per row of W. It is the only place where parameters touch activations (apart from the embedding lookups), so every learned transformation in the network is linear, and all non-linearity comes from softmax, ReLU and normalisation.

softmax turns a vector of real scores $z \in \mathbb{R}^n$ into a probability vector,

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}} \quad \text{for any constant } m. \tag{2}$$

Subtracting $m = \max_j z_j$ changes nothing mathematically but keeps every exponent ≤ 0, so `exp` cannot overflow. Note that `max_val` is a plain float read from `.data`; the subtraction is therefore a constant shift in the graph and does not need its own gradient. (Its gradient would cancel exactly anyway.)

rmsnorm (Zhang and Sennrich, 2019) rescales a vector to unit root-mean-square:

$$\text{rmsnorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \varepsilon}}, \qquad \varepsilon = 10^{-5}. \tag{3}$$

Compared with LayerNorm (Ba et al., 2016) it drops the mean subtraction and, in microgpt, the learned per-dimension gain as well. Its job is to keep activation magnitudes from drifting as signals flow through the network, which stabilises optimisation.

6.2 The model function

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
```

```
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len
    ↪ (k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits
```

6.3 Embeddings

A network cannot do arithmetic on the symbol "5". Each token id selects a learned row of the table `wte` $\in \mathbb{R}^{V\times d}$, and each position selects a row of `wpe` $\in \mathbb{R}^{T_{\max}\times d}$. The two vectors are added:

$$x_i^{(0)} = \mathrm{rmsnorm}\left(\mathrm{wte}[t_i] + \mathrm{wpe}[i]\right). \tag{4}$$

The result encodes *what* the token is and *where* it is. Because the rest of the network is permutation-invariant in the cached positions (attention is a weighted sum), the position embedding is the only source of order information. The original Transformer used fixed sinusoids (Vaswani et al., 2017); GPT-1 and GPT-2 learned the table as here (Radford et al., 2018, 2019); most current models use rotary embeddings (RoPE, Su et al. 2021) which inject *relative* position into q and k instead.

The source comments that normalising the embedding sum is "not redundant due to backward pass via the residual connection". The pre-norm inside the attention block only normalises the copy of x that enters the branch; the copy that continues down the residual path is untouched. Normalising once at the start therefore fixes the scale at which the residual stream begins, which matters for both the forward values and the gradients that flow back along the stream.

6.4 Attention

The current normalised vector x is projected into a query, a key and a value:

$$q = W_q x, \qquad k = W_k x, \qquad v = W_v x, \qquad W_q, W_k, W_v \in \mathbb{R}^{d\times d}. \tag{5}$$

Karpathy's gloss: the query asks "what am I looking for?", the key says "what do I contain?", the value says "what do I offer if selected?". The new k and v are appended to the per-layer cache, so after processing positions $0, \ldots, i$ the cache holds $k_0, \ldots, k_i$ and $v_0, \ldots, v_i$.

Each of the H heads works on its own slice of the d coordinates. For head h with slice $S_h = \{hd_h, \ldots, (h +$

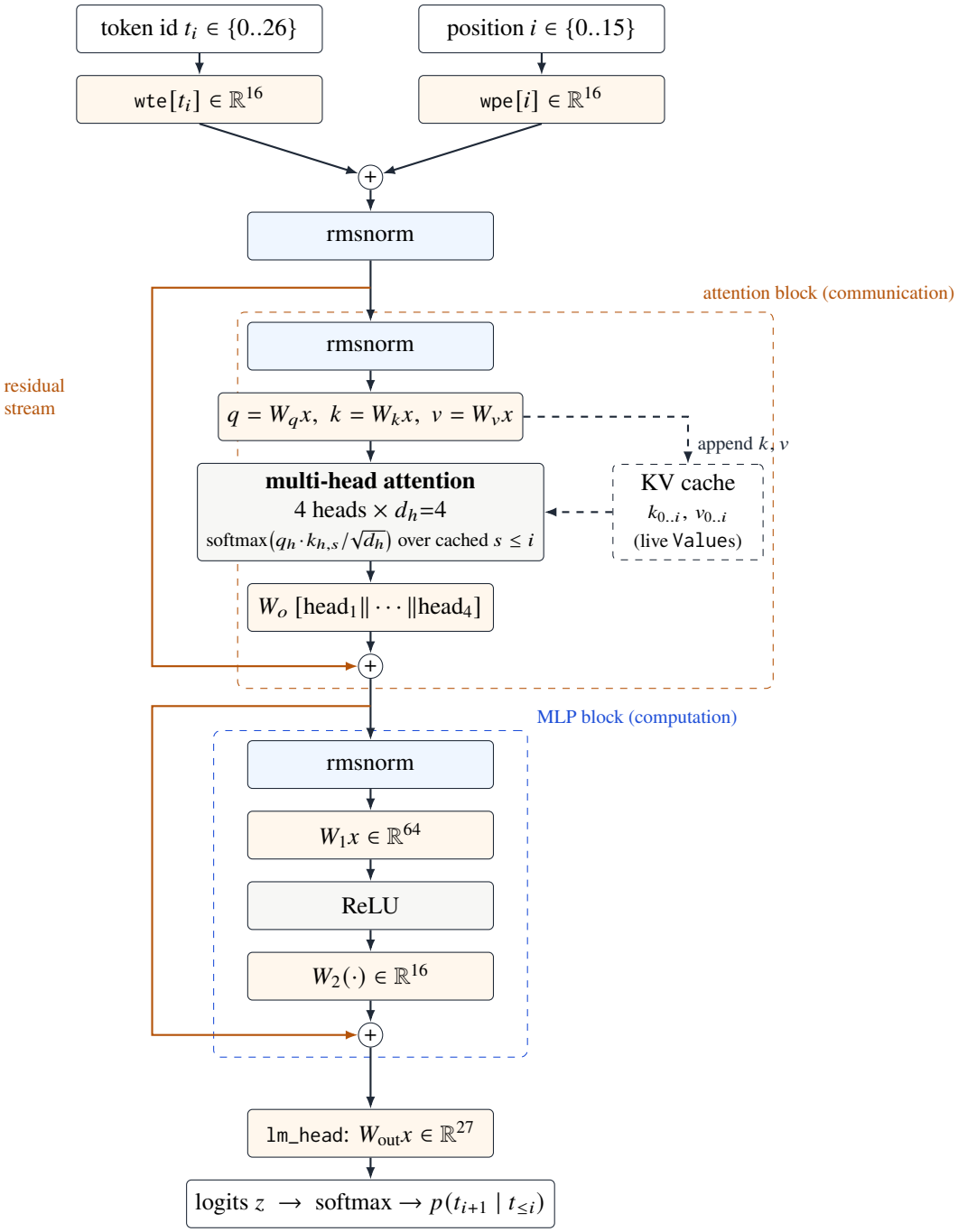


Figure 3: One call of `gpt(token_id, pos_id, keys, values)` for $L = 1$. Orange boxes hold parameters, blue boxes are parameter-free normalisations. The two orange side-arrows are the residual connections. With $L > 1$ the dashed region repeats L times; the KV cache has one list of keys and one of values per layer.

$1)d_h - 1\}$:

$$a_{h,s} = \frac{q[S_h] \cdot k_s[S_h]}{\sqrt{d_h}}, \qquad s = 0, \ldots, i, \tag{6}$$

$$\alpha_h = \mathrm{softmax}(a_{h,0}, \ldots, a_{h,i}), \tag{7}$$

$$\mathrm{head}_h = \sum_{s=0}^{i} \alpha_{h,s}\, v_s[S_h] \in \mathbb{R}^{d_h}. \tag{8}$$

The head outputs are concatenated into a vector of length d and projected by W_o. In matrix form this is the familiar

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}}\Big)V \tag{9}$$

of Vaswani et al. (2017), specialised to a single query row.

Where is the causal mask? Vectorised implementations compute all $T \times T$ scores at once and then mask out $s > i$ with $-\infty$. microgpt needs no mask: at position i the cache simply does not yet contain anything from the future. Causality is a consequence of processing tokens in order.

Why $\sqrt{d_h}$? If q and k have independent coordinates of unit variance, their dot product has variance d_h. Dividing by $\sqrt{d_h}$ restores unit variance, so the softmax is neither flat nor saturated at initialisation (Vaswani et al., 2017). With $d_h = 4$ the factor is $1/2$.

Why several heads? Each head has its own W_q, W_k, W_v slices and so can implement a different pattern of lookup ("the previous letter", "the most recent vowel", "the first letter of the name"). Four heads of width four cost exactly the same as one head of width sixteen; the split buys diversity, not capacity.

Key idea: attention is communication

The attention block is the *only* place where position i reads information from positions $0, \ldots, i-1$. Everything else in the network acts on one position at a time. Karpathy's summary: the Transformer interleaves *communication* (attention) with *computation* (MLP), both wired into a shared residual stream.

6.5 The KV cache, during training

Readers who know Transformers from the serving side associate the KV cache with inference. microgpt uses it during training too, and the post argues that the cache is "conceptually always there": a batched implementation computes all keys and values for a sequence in one matrix multiply and the cache is simply hidden inside that tensor. Because microgpt processes one token at a time, the cache becomes explicit.

The subtle point is that the cached entries are *live* `Value` nodes. When the loss at position 7 depends on k_2 and v_2, the backward pass flows gradient into the parameters that produced them, through the cache. In production inference the cached tensors are detached; in production training there is no explicit cache but the same gradient paths exist inside the attention kernel. The memory cost of the cache, $2\,L\,T\,d$ numbers per sequence, is what motivates multi-query and grouped-query attention (Ainslie et al., 2023; Shazeer, 2019) and paged cache management in vLLM (Kwon et al., 2023); Pope et al. (2022) give the accounting.

6.6 The MLP block

$$\mathrm{MLP}(x) = W_2\ \mathrm{relu}(W_1 x), \qquad W_1 \in \mathbb{R}^{4d\times d},\ W_2 \in \mathbb{R}^{d\times 4d}. \tag{10}$$

Project up by a factor of four, apply an elementwise non-linearity, project back down. The 4× widening comes from the original Transformer and GPT-2. The MLP is where per-position "thinking" happens: having gathered context through attention, each position transforms its own vector without reference to any other.

GPT-2 used GELU (Hendrycks and Gimpel, 2016); microgpt uses ReLU (Glorot et al., 2011; Nair and Hinton, 2010) because its gradient is a single comparison. Current models mostly use gated variants such as SwiGLU (Shazeer, 2020; Touvron et al., 2023); squared ReLU (So et al., 2021) is another popular simplification used in nanochat (Karpathy, 2025).

6.7 Residual connections and pre-normalisation

Both blocks add their output to their input: $x \leftarrow x + f(\mathrm{rmsnorm}(x))$. The identity path means that gradients reach early layers undiminished, which is what makes 100-layer networks trainable (He et al., 2016). Placing the normalisation *before* each block (pre-LN) rather than after it, as GPT-2 did, makes training markedly more stable (Xiong et al., 2020). The residual-stream picture, in which blocks *read from* and *write to* a shared vector, is developed in Elhage et al. (2021).

6.8 Output

Finally `lm_head` maps the hidden state to $V = 27$ logits. Higher logit means the model considers that token more likely to come next. The logits are not probabilities; the softmax that converts them is applied by the caller, inside the loss during training and inside the sampler at inference.

6.9 Cost of one token

Counting multiply–adds per position for $L = 1$: $4d^2$ for the attention projections, $8d^2$ for the MLP, Vd for the head, plus about $2(i{+}1)d$ for the attention scores and weighted sum at position i. For $d = 16$ this is about 3,500 multiply–adds, essentially 2× the number of non-embedding parameters. The rule of thumb "$\approx 2N$ FLOPs per token for a forward pass and $\approx 6N$ for a training step" (Kaplan et al., 2020) already holds for microgpt.

7 Training loop: loss, gradients and Adam

```
# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
```

```
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')
```

Each step performs five actions: pick a document, run the model forward over it, compute a loss, backpropagate, update.

7.1 Next-token prediction

The document "emma" becomes `[BOS, e, m, m, a, BOS]`, a sequence $t_0, \ldots, t_5$. Position i sees token t_i (and, through the cache, everything before it) and must predict t_{i+1}. There are $n = \min(T_{\max}, |\text{tokens}| - 1)$ such prediction problems per document; for "emma" $n = 5$. Because the longest name has 15 letters, the clamp to $T_{\max} = 16$ never truncates a name.

7.2 Cross-entropy loss

At position i the model's softmax output is a distribution $p(\cdot \mid t_{\le i})$ over the 27 tokens, and the loss is the negative log-probability of the token that actually came next:

$$\ell_i = -\log p(t_{i+1} \mid t_{\le i}), \qquad \mathcal{L} = \frac{1}{n} \sum_{i=0}^{n-1} \ell_i. \tag{11}$$

Why negative log probability?

Three equivalent views.

- **Surprise.** $-\log p$ is 0 when the model was certain and right, and grows without bound as the probability assigned to the truth shrinks. It is the "surprisal" of information theory (Cover and Thomas, 2006; Shannon, 1948).
- **Maximum likelihood.** Minimising $\sum_i -\log p(t_{i+1} \mid t_{\le i})$ over a corpus is maximising the log-likelihood of the corpus under the model, since $\log p(\text{document}) = \sum_i \log p(t_{i+1} \mid t_{\le i})$ by the chain rule of probability.
- **Cross-entropy.** With the one-hot target distribution y, $-\sum_k y_k \log p_k = -\log p_{\text{target}}$ is the cross-entropy $H(y, p)$, which exceeds the entropy of the data by exactly the KL divergence from data to model (Bishop, 2006; Goodfellow et al., 2016).

Reference values. A model that knows nothing assigns $p = 1/27$ to everything, giving

$$\mathcal{L}_{\text{uniform}} = -\log \tfrac{1}{27} = \ln 27 \approx 3.296. \tag{12}$$

The first printed loss, 3.366, is close to this; the random initial weights make it slightly worse than uniform. The lowest possible loss is 0. After 1,000 steps the smoothed loss is about 2.3 (fig. 4); Karpathy quotes ≈ 2.37. Two natural rescalings: the *perplexity* $e^{\mathcal{L}}$ falls from 27 to about 10, meaning the model is as uncertain as if choosing uniformly among ten letters; and $\mathcal{L}/\ln 2$ gives bits per character, about 3.4 at the end versus 4.75 for a uniform guess.

The gradient at the logits. For completeness, the combined softmax–cross-entropy gradient is remarkably simple:

$$\frac{\partial \ell}{\partial z_k} = p_k - y_k, \tag{13}$$

that is, "predicted probability minus one-hot target". microgpt never writes this down; the autograd engine derives it from the `exp`, `log`, division and sum nodes in table 1. Section 13 asks you to verify the identity numerically.

7.3 Batch size one, and what it means for the loss curve

Each step uses exactly one document. In the language of Robbins and Monro (1951) this is stochastic gradient descent with a minibatch of one sequence (of up to 16 tokens). Two consequences:

- The printed loss is noisy. A step on a short, common name such as "ann" yields a lower loss than a step on "christopher", regardless of learning. Figure 4 shows raw and smoothed curves from our run; the last single-document loss was 2.65 while the trailing 50-step mean was 2.32.
- The model sees only the first 1,000 of the 32,033 shuffled names, about 3% of one epoch. Every training document is new, so the training loss is effectively a held-out loss. There is no overfitting to speak of, and no need for a validation split at this scale.

7.4 Backward

One call to `loss.backward()` runs the algorithm of section 4 over the whole graph, from the loss through the averaging, the per-position softmaxes and the model, including the cached keys and values of earlier positions, down into every parameter. Afterwards each parameter's `.grad` is $\partial \mathcal{L}/\partial \theta$. In our measurements a step on a 12-letter name builds a graph of about 10^5 nodes; forward takes a quarter of a second and backward a tenth.

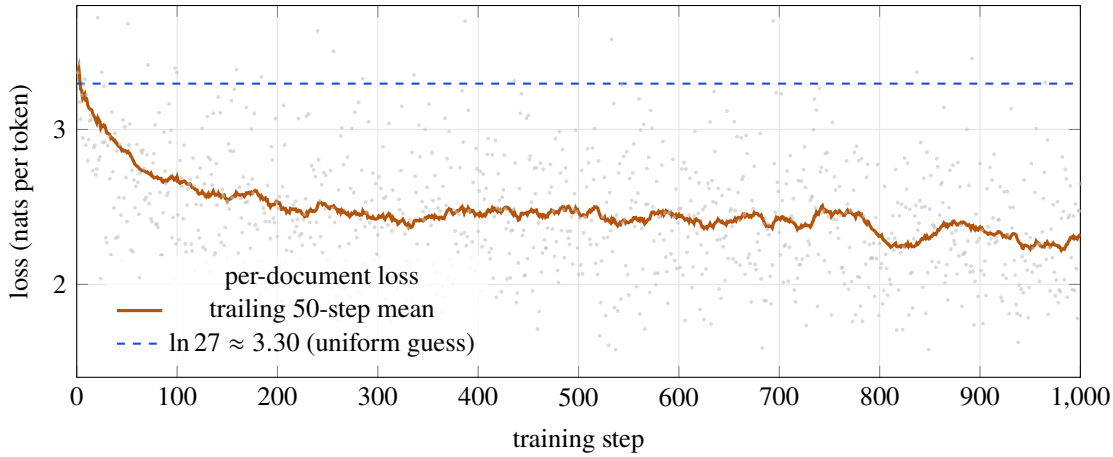


Figure 4: Loss over the 1,000 training steps of `microgpt.py` with the default seed. Each grey dot is one document; the orange line is a trailing average. The spread of the dots is the noise of batch size one, not instability.

7.5 Adam

Plain gradient descent would update $\theta \leftarrow \theta - \eta\, g$ with $g = \partial \mathcal{L}/\partial\theta$. Adam (Kingma and Ba, 2015) keeps two exponential moving averages per parameter and uses their ratio:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \tag{14}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{15}$$

$$\theta_t = \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{16}$$

- m is **momentum** (Polyak, 1964): a running mean of gradients that smooths out the step-to-step noise of single documents, "like a rolling ball".
- v is a running mean of squared gradients, the idea of RMSProp (Tieleman and Hinton, 2012). Dividing by $\sqrt{\hat{v}}$ gives every parameter its own effective learning rate: parameters whose gradients are consistently large take smaller steps, rarely-updated parameters (such as embedding rows of rare letters) take larger ones.
- $\hat{m}, \hat{v}$ are **bias corrections**. Both buffers start at zero and are therefore biased toward zero during the first few dozen steps; dividing by $1-\beta^t$ undoes this exactly in expectation.
- $\epsilon = 10^{-8}$ prevents division by zero.

microgpt uses $\beta_1 = 0.85$, $\beta_2 = 0.99$ rather than the paper's defaults 0.9, 0.999: shorter memories suit a run of only 1,000 noisy steps. The learning rate $\eta_t = 0.01\,(1 - t/1000)$ decays linearly to zero, a simple instance of the schedules surveyed by Loshchilov and Hutter (2017); large models typically use a warmup followed by cosine or linear decay. Weight decay (Loshchilov and Hutter, 2019), universal at scale, is omitted here.

After the update every gradient is reset to zero. Without this, the next step's `+=` accumulation would add new gradients on top of stale ones.

8 Inference: sampling new names

```
# Inference: may the model babble back to us
```

```
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

With parameters frozen, generation is the same forward function in a loop. Start with an empty cache and the `BOS` token; compute logits; convert to probabilities; draw one token at random according to those probabilities; feed it back as the next input; stop when `BOS` is drawn or after $T_{\max}$ tokens. This is *autoregressive* or *ancestral* sampling: the model's own output becomes its input.

8.1 Temperature

Dividing the logits by T before the softmax gives

$$p_k^{(T)} = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}. \tag{17}$$

$T = 1$ samples from the learned distribution. $T < 1$ sharpens it: the ratio between the top two probabilities is raised to the power $1/T$, so the model becomes more conservative. As $T \to 0$ the softmax becomes a one-hot on the arg-max, which is greedy decoding. $T > 1$ flattens the distribution toward uniform and produces more varied but less coherent output. The name and formula come from the Boltzmann distribution of statistical physics by way of Ackley et al. (1985); Hinton et al. (2015) discuss it in the modern setting. Truncation methods such as top-k (Fan et al., 2018) and nucleus sampling (Holtzman et al., 2020) are the usual companions of temperature in production and are a one-line addition here (section 13).

8.2 What the samples show

With the default seed the twenty samples are *kamon, ann, karai, jaire, vialan, karia, yeran, anna, areli, kaina, konna, keylen, liole, alerin, earan, lenne, kana, lara, alela, anton*. They are pronounceable, alternate consonants and vowels, end at plausible lengths, and a few (*ann*, *anna*, *lara*, *anton*) are real names the model has rediscovered rather than copied; most are not in the training set at all. This is the whole phenomenon of "hallucination": the model produces statistically plausible sequences with no notion of whether they are real.

9 Running it

```
python microgpt.py
```

Expected output (the loss line is overwritten in place in the gist version; the blog shows every step):

```
num docs: 32033
vocab size: 27
```

```
num params: 4192
step    1 / 1000 | loss 3.3660
step    2 / 1000 | loss 3.4243
step    3 / 1000 | loss 3.1778
...
step 1000 / 1000 | loss 2.6497

--- inference (new, hallucinated names) ---
sample  1: kamon
sample  2: ann
sample  3: karai
...
```

Measurements for these notes. Running the unmodified gist on a 2026 laptop with CPython took 3 minutes 24 seconds (Karpathy reports about one minute on his machine). Instrumenting the `Value` constructor gives the graph sizes in table 3: roughly 8,000 nodes per token, dominated by the 16×16 and 16×64 linear layers, each of which creates one multiplication and one addition node per weight.

Document	n (predictions)	`Value` nodes	forward (s)	backward (s)
emma	5	39,703	0.055	0.038
isabella	9	73,263	0.161	0.069
christopher	12	99,483	0.251	0.099

Table 3: Size of the computation graph for one training step on three names at initialisation.

Things to try. Increase `num_steps`; raise `n_embd`, `n_layer` or `n_head`; change `temperature`; replace `input.txt` with another list of short documents. The Colab notebook (Karpathy, 2026c) lets you do this without installing anything.

10 Progression: building the program in six steps

Karpathy recommends reading the program as layers of an onion, using the revision history of `build_microgpt.py` (Karpathy, 2026a):

File	What it adds
`train0.py`	Bigram count table: no neural network, no gradients. Count how often each letter follows each other letter and sample from the normalised counts.
`train1.py`	A multilayer perceptron with *manual* gradients, checked numerically and analytically, trained by SGD.
`train2.py`	Autograd: the `Value` class replaces the hand-written gradients.
`train3.py`	Position embeddings, single-head attention, RMSNorm and residual connections.
`train4.py`	Multi-head attention and the layer loop: the full GPT architecture.
`train5.py`	Adam replaces SGD. This file is `train.py`, i.e. microgpt.

Table 4: The suggested order in which to build (or read) microgpt.

This mirrors the historical development: count-based n-gram models (Jurafsky and Martin, 2025; Shannon, 1951), neural language models (Bengio et al., 2003), backpropagation (Rumelhart et al., 1986), attention (Bahdanau et al., 2015; Vaswani et al., 2017) and adaptive optimizers (Kingma and Ba, 2015). The bigram stage is especially instructive: it already produces name-like output, and its table of 27×27 counts is exactly

what a one-position model with no attention could learn. Everything after `train0.py` is about using *more* context than the previous letter.

11 From microgpt to a production LLM

Karpathy's closing argument is that nothing between microgpt and ChatGPT changes the algorithm; the changes make it work at scale. We follow the sections of the program in order and attach the main references.

Component	microgpt	Production
Data	32K names, one list	Trillions of tokens of web, books, code; deduplicated, quality-filtered, domain-mixed (Gao et al., 2020; Lee et al., 2022; Penedo et al., 2024; Raffel et al., 2020)
Tokenizer	27 characters	Byte-level BPE, ~100K subword tokens (OpenAI, 2022; Radford et al., 2019; Sennrich et al., 2016)
Autograd	Scalar `Value` in Python	Tensor autograd on GPUs/TPUs; fused kernels such as FlashAttention (Dao, 2024; Dao et al., 2022; Paszke et al., 2019)
Architecture	4,192 params, $d = 16$, $L = 1$	10^{11}–10^{12} params, $d > 10^4$, $L > 100$; RoPE, GQA, gated MLPs, Mixture of Experts (Ainslie et al., 2023; DeepSeek-AI, 2024; Fedus et al., 2022; Shazeer, 2020; Su et al., 2021)
Training	One document per step	Millions of tokens per step, gradient accumulation, mixed precision, thousands of GPUs for months (Micikevicius et al., 2018)
Optimisation	Adam, linear decay	Tuned AdamW with warmup and weight decay, bf16/fp8 numerics, scaling laws to size the run (Hoffmann et al., 2022; Kaplan et al., 2020; Loshchilov and Hutter, 2019)
Post-training	none	SFT on conversations, then RL from feedback (Christiano et al., 2017; Ouyang et al., 2022; Rafailov et al., 2023; Schulman et al., 2017; Shao et al., 2024)
Inference	One sample at a time	Batching, paged KV cache, speculative decoding, quantisation, multi-GPU sharding (Chen et al., 2023; Dettmers et al., 2022; Frantar et al., 2023; Kwon et al., 2023; Leviathan et al., 2023; Yu et al., 2022)

Table 5: What changes on the way to a frontier model, with entry points into the literature.

11.1 Data

Production corpora are built from web crawls, books and code, then deduplicated (which measurably improves models, Lee et al. 2022), filtered for quality and mixed across domains in tuned proportions. The microgpt analogue is `names.txt`.

11.2 Tokenizer

Subword tokenization (section 6 discussed BPE) lets each position carry roughly four characters of English on average, which multiplies the effective context length and reduces the number of sequential steps per document.

11.3 Autograd and hardware

The mathematics of section 4 is unchanged, but operations act on tensors and a single backward node may correspond to a fused CUDA kernel. FlashAttention (Dao et al., 2022) computes exact attention without materialising the $T \times T$ score matrix, by tiling it through fast on-chip memory; this is the kind of "efficiency" Karpathy has in mind.

11.4 Architecture

The residual stream with alternating attention and MLP blocks is preserved. The main substitutions are RoPE for learned position embeddings (Su et al., 2021), grouped-query attention to shrink the KV cache (Ainslie et al., 2023), gated activations (Shazeer, 2020) and, in many recent models, Mixture-of-Experts MLPs in which each token is routed to a few of many expert MLPs (DeepSeek-AI, 2024; Fedus et al., 2022; Shazeer et al., 2017). Phuong and Hutter (2022) give pseudocode for all these variants in a notation close to microgpt's.

11.5 Training and optimisation

Batches of millions of tokens, gradient accumulation across devices, and mixed precision (Micikevicius et al., 2018) replace the single-document step. Optimisation becomes a discipline of its own: learning rate, warmup, decay, weight decay and the Adam betas interact with model and batch size, and mistakes cost millions of dollars of compute. Scaling laws (Kaplan et al., 2020) and the compute-optimal allocation of Hoffmann et al. (2022) (roughly 20 training tokens per parameter) tell teams how big to make the model for a given budget, and small-scale experiments are used to predict the right settings before a full run.

11.6 Post-training

The pretrained model is a document completer. Two further stages make it a chatbot. *Supervised fine-tuning* swaps the documents for curated conversations and keeps running the same loop; algorithmically nothing changes. *Reinforcement learning* has the model generate responses, scores them (by humans, a judge model or a program) and updates toward high-scoring outputs (Christiano et al., 2017; Ouyang et al., 2022; Schulman et al., 2017); direct preference optimisation (Rafailov et al., 2023) and GRPO (Shao et al., 2024) are recent variants. Karpathy's framing: the model is still training on documents, except that the documents are now written by the model itself. nanochat (Karpathy, 2025) runs the full pipeline end to end at small scale.

11.7 Inference

Serving millions of users requires continuous batching (Yu et al., 2022), paged KV-cache memory management (Kwon et al., 2023), speculative decoding in which a small draft model proposes tokens that the large model verifies in parallel (Chen et al., 2023; Leviathan et al., 2023), quantisation to 8 or 4 bits (Dettmers et al., 2022; Frantar et al., 2023), and sharding the model across many GPUs. Fundamentally it is still the loop of section 8.

Key idea: the essence

"All of these are important engineering and research contributions but if you understand microgpt, you understand the algorithmic essence." (Karpathy, 2026b)

12 Frequently asked questions

The post ends with a FAQ. We paraphrase each answer and add a pointer.

Does the model "understand" anything? Mechanically, no magic is involved: the model is a function from input tokens to a distribution over the next token, and training adjusts its parameters to make the correct token more probable. Whether that constitutes understanding is a philosophical question; the mechanism is fully contained in the 200 lines.

Why does it work? Thousands of parameters are nudged a little every step in the direction that lowers the loss. Over many steps they settle into values that encode the statistics of the data: names usually start with a consonant, q is followed by u, three consonants in a row are rare. The model learns no rules, only a distribution that happens to reflect them.

How is this related to ChatGPT? It is the same loop (predict next token, sample, repeat), scaled up enormously and post-trained to be conversational. System prompt, user message and reply are all tokens in one sequence.

What are "hallucinations"? Samples from a distribution over plausible sequences. The model has no notion of truth; *karia* is to microgpt what a confidently stated false fact is to a chatbot.

Why is it so slow? One scalar at a time in interpreted Python. A GPU processes millions of scalars in parallel; the same mathematics runs orders of magnitude faster.

Can it generate better names? Train longer, make the model bigger, or use more data. These are the same three knobs that scaling laws (Hoffmann et al., 2022; Kaplan et al., 2020) are about.

What if I change the dataset? The model learns whatever is in the file: city names, Pokémon, English words, short poems. No other line changes.

13 Exercises

Exercise 1 (Parameter count). Without running the code, derive 4,192 from the hyperparameters. Then write a formula for the parameter count as a function of V, d, L and $T_{\max}$, and evaluate it for GPT-2 small ($V = 50257$, $d = 768$, $L = 12$, $T_{\max} = 1024$), remembering that GPT-2 has biases, LayerNorm gains and tied embeddings.

Exercise 2 (Gradient check). Pick a parameter θ, record its `.grad` after `loss.backward()`, then perturb `.data` by $\pm h$ with $h = 10^{-4}$, recompute the loss, and compare the centred finite difference $(\mathcal{L}(\theta + h) - \mathcal{L}(\theta - h))/2h$ with the stored gradient. This is what `train1.py` did by hand.

Exercise 3 (Softmax–cross-entropy). Verify numerically that $\partial \ell / \partial z_k = p_k - y_k$ by building a small graph with `Values` and calling `backward()`. Then derive it by hand from $\ell = -z_y + \log \sum_j e^{z_j}$.

Exercise 4 (Bigram baseline). Implement `train0.py`: a 27×27 table of counts of (current, next) pairs over all names, smoothed by adding 1, normalised row-wise. Compute its average per-token loss on the data and compare with microgpt's ≈ 2.3. Sample twenty names.

Exercise 5 (Causality). Argue from the code that position i can never depend on tokens t_j with $j > i$. Then modify the training loop to compute all positions' logits first and the loss afterwards; does the result change? Why not?

Exercise 6 (Attention by hand). For a two-token input with $d_h = 2$, choose small integer q, k_0, k_1, v_0, v_1 and compute eqs. (6) to (8) by hand. Check with the code.

Exercise 7 (SGD versus Adam). Replace the Adam update by `p.data -= lr_t * p.grad` and find the learning rate that gives the best loss after 1,000 steps. Compare curves. Then restore Adam and try $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Exercise 8 (Learning-rate schedule). Replace linear decay by a constant rate, then by cosine decay $\eta_t = \frac{\eta}{2}(1 + \cos(\pi t/N))$. Plot smoothed losses.

Exercise 9 (Minibatches). Average the loss over B documents before calling `backward()`. Keep the total number of documents seen fixed. How do noise and wall-clock time change?

Exercise 10 (Validation). Hold out 10% of the names and evaluate the mean loss on them every 100 steps (no `backward()` needed). Then train for 50,000 steps, so that the model sees each name once or more, and look for a gap between training and held-out loss.

Exercise 11 (Decoding). Implement greedy decoding ($T \to 0$), top-k sampling and nucleus sampling. Generate twenty names with each and comment on diversity versus plausibility.

Exercise 12 (Architectural variants). One at a time: (a) add bias vectors to every `linear`; (b) tie `lm_head` to `wte`; (c) replace ReLU by squared ReLU; (d) add a learned gain vector to `rmsnorm`; (e) use $n_{\text{layer}} = 2$ or 4. For each, note the parameter count and the smoothed final loss.

Exercise 13 (RoPE). Remove `wpe` and instead rotate each pair of coordinates in q and k by an angle proportional to the position, following Su et al. (2021). Check that attention scores now depend only on relative position.

Exercise 14 (Graph size). Instrument `Value.__init__` with a counter and reproduce table 3. Derive an approximate closed form for the node count as a function of n, d, V and L.

Exercise 15 (New data). Train on a different list of short strings (words, place names, chemical element names). What changes in the loss floor, and why might a dataset of English words reach a lower loss per character than names?

A Complete source of `microgpt.py`

The gist as retrieved on October 11, 2026 (Karpathy, 2026d), reproduced for study. Line numbers are added.

```python
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forward pass
        self.grad = 0                   # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
```

```
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad

# Initialize the parameters, to store the knowledge of the model
n_layer = 1     # depth of the transformer neural network (number of layers)
n_embd = 16     # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4      # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f"num params: {len(params)}")

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm → rmsnorm, no biases, GeLU → ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
```

```
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
```

```
200     print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

References

Ackley, D. H., G. E. Hinton, and T. J. Sejnowski (1985). "A Learning Algorithm for Boltzmann Machines". In: *Cognitive Science* 9.1. Origin of the "temperature" softmax in neural networks, pp. 147–169.

Ainslie, J., J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of EMNLP*. arXiv:2305.13245.

Ba, J. L., J. R. Kiros, and G. E. Hinton (2016). "Layer Normalization". In: *arXiv preprint arXiv:1607.06450*.

Bahdanau, D., K. Cho, and Y. Bengio (2015). "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations (ICLR)*. arXiv:1409.0473.

Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). "Automatic Differentiation in Machine Learning: a Survey". In: *Journal of Machine Learning Research* 18.153, pp. 1–43.

Bengio, Y., R. Ducharme, P. Vincent, and C. Jauvin (2003). "A Neural Probabilistic Language Model". In: *Journal of Machine Learning Research* 3, pp. 1137–1155.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Brown, T. B. et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. arXiv:2005.14165.

Chen, C., S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper (2023). "Accelerating Large Language Model Decoding with Speculative Sampling". In: *arXiv preprint arXiv:2302.01318*.

Christiano, P. F., J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei (2017). "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03741.

Cover, T. M. and J. A. Thomas (2006). *Elements of Information Theory*. 2nd ed. Wiley.

Dao, T. (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*. arXiv:2307.08691.

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2205.14135.

DeepSeek-AI (2024). "DeepSeek-V3 Technical Report". In: *arXiv preprint arXiv:2412.19437*. Large-scale MoE, multi-head latent attention, FP8 training.

Dettmers, T., M. Lewis, Y. Belkada, and L. Zettlemoyer (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2208.07339.

Elhage, N. et al. (2021). *A Mathematical Framework for Transformer Circuits*. The "residual stream" view of the Transformer. URL: https://transformer-circuits.pub/2021/framework/index.html (visited on 10/11/2026).

Fan, A., M. Lewis, and Y. Dauphin (2018). "Hierarchical Neural Story Generation". In: *Proceedings of ACL*. arXiv:1805.04833; top-k sampling.

Fedus, W., B. Zoph, and N. Shazeer (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39.

Frantar, E., S. Ashkboos, T. Hoefler, and D. Alistarh (2023). "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*. arXiv:2210.17323.

Gage, P. (1994). "A New Algorithm for Data Compression". In: *C Users Journal* 12.2, pp. 23–38.

Gao, L. et al. (2020). "The Pile: An 800GB Dataset of Diverse Text for Language Modeling". In: *arXiv preprint arXiv:2101.00027*.

Glorot, X. and Y. Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*.

Glorot, X., A. Bordes, and Y. Bengio (2011). "Deep Sparse Rectifier Neural Networks". In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*.

Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. Free online at https://www.deeplearningbook.org. MIT Press.

Graves, A. (2013). "Generating Sequences With Recurrent Neural Networks". In: *arXiv preprint arXiv:1308.0850*.

Griewank, A. and A. Walther (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. SIAM.

He, K., X. Zhang, S. Ren, and J. Sun (2016). "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1512.03385, pp. 770–778.

Hendrycks, D. and K. Gimpel (2016). "Gaussian Error Linear Units (GELUs)". In: *arXiv preprint arXiv:1606.08415*.

Hinton, G., O. Vinyals, and J. Dean (2015). "Distilling the Knowledge in a Neural Network". In: *arXiv preprint arXiv:1503.02531*. Clear discussion of softmax temperature.

Hoffmann, J. et al. (2022). "Training Compute-Optimal Large Language Models". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.15556; the "Chinchilla" scaling laws.

Holtzman, A., J. Buys, L. Du, M. Forbes, and Y. Choi (2020). "The Curious Case of Neural Text Degeneration". In: *International Conference on Learning Representations (ICLR)*. arXiv:1904.09751; nucleus (top-p) sampling.

Jurafsky, D. and J. H. Martin (2025). *Speech and Language Processing*. 3rd ed. draft. https://web.stanford.edu/~jurafsky/slp3/; chapters on n-grams, neural LMs, Transformers. Online.

Kaplan, J. et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*.

Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*. URL: https://karpathy.github.io/2015/05/21/rnn-effectiveness/ (visited on 10/11/2026).

Karpathy, A. (2020). *micrograd: a tiny scalar-valued autograd engine and a neural net library on top of it*. URL: https://github.com/karpathy/micrograd (visited on 10/11/2026).

Karpathy, A. (2022a). *makemore: an autoregressive character-level language model for making more things*. Source of the `names.txt` dataset used by microgpt. URL: https://github.com/karpathy/makemore (visited on 10/11/2026).

Karpathy, A. (2022b). *nanoGPT: the simplest, fastest repository for training/finetuning medium-sized GPTs*. URL: https://github.com/karpathy/nanoGPT (visited on 10/11/2026).

Karpathy, A. (2022c). *The spelled-out intro to neural networks and backpropagation: building micrograd*. Video lecture, approx. 2.5 hours. URL: https://www.youtube.com/watch?v=VMj-3S1tku0 (visited on 10/11/2026).

Karpathy, A. (2024a). *Let's build the GPT Tokenizer*. URL: https://www.youtube.com/watch?v=zduSFxRajkE (visited on 10/11/2026).

Karpathy, A. (2024b). *llm.c: LLM training in simple, raw C/CUDA*. URL: https://github.com/karpathy/llm.c (visited on 10/11/2026).

Karpathy, A. (2025). *nanochat: the best ChatGPT that $100 can buy*. Full pretraining, SFT, RL and serving pipeline in one small repo. URL: https://github.com/karpathy/nanochat (visited on 10/11/2026).

Karpathy, A. (2026a). *build_microgpt.py: builds up microgpt.py piece by piece*. GitHub gist; the Revisions tab holds train0.py through train5.py. URL: https://gist.github.com/karpathy/561ac2de12a47cc06a23691e1be9543a (visited on 10/11/2026).

Karpathy, A. (Feb. 2026b). *microgpt*. Blog post; the primary source for this course. URL: https://karpathy.github.io/2026/02/12/microgpt/ (visited on 10/11/2026).

Karpathy, A. (2026c). *microgpt Google Colab notebook*. URL: https://colab.research.google.com/drive/1vyN5zo6rqUp_dYNbT4Yrco66zuWCZKoN (visited on 10/11/2026).

Karpathy, A. (2026d). *microgpt.py: the most atomic way to train and run inference for a GPT in pure, dependency-free Python*. GitHub gist, 200 lines. URL: https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95 (visited on 10/11/2026).

Kingma, D. P. and J. Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

Kudo, T. and J. Richardson (2018). "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing". In: *Proceedings of EMNLP: System Demonstrations*. arXiv:1808.06226.

Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. arXiv:2309.06180; vLLM.

Lee, K., D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini (2022). "Deduplicating Training Data Makes Language Models Better". In: *Proceedings of ACL*. arXiv:2107.06499.

Leviathan, Y., M. Kalman, and Y. Matias (2023). "Fast Inference from Transformers via Speculative Decoding". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. arXiv:2211.17192.

Linnainmaa, S. (1976). "Taylor Expansion of the Accumulated Rounding Error". In: *BIT Numerical Mathematics* 16.2. The earliest published form of reverse-mode automatic differentiation, pp. 146–160.

Loshchilov, I. and F. Hutter (2017). "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations (ICLR)*. arXiv:1608.03983; cosine learning-rate schedules.

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*. arXiv:1711.05101; AdamW.

Micikevicius, P. et al. (2018). "Mixed Precision Training". In: *International Conference on Learning Representations (ICLR)*. arXiv:1710.03740.

Nair, V. and G. E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*.

OpenAI (2022). *tiktoken: a fast BPE tokeniser for use with OpenAI's models*. URL: https://github.com/openai/tiktoken (visited on 10/11/2026).

Ouyang, L. et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.02155; InstructGPT: SFT + RLHF.

Paszke, A. et al. (2017). "Automatic Differentiation in PyTorch". In: *NIPS 2017 Autodiff Workshop*.

Paszke, A. et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*.

Penedo, G., H. Kydlíček, L. B. Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf (2024). "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale". In: *arXiv preprint arXiv:2406.17557*.

Phuong, M. and M. Hutter (2022). "Formal Algorithms for Transformers". In: *arXiv preprint arXiv:2207.09238*. Pseudocode for every Transformer variant; an excellent companion to the code.

Polyak, B. T. (1964). "Some Methods of Speeding Up the Convergence of Iteration Methods". In: *USSR Computational Mathematics and Mathematical Physics* 4.5, pp. 1–17.

Pope, R. et al. (2022). "Efficiently Scaling Transformer Inference". In: *arXiv preprint arXiv:2211.05102*. Careful accounting of KV-cache memory and bandwidth.

Press, O. and L. Wolf (2017). "Using the Output Embedding to Improve Language Models". In: *Proceedings of EACL*. arXiv:1608.05859; microgpt deliberately does *not* tie wte and lm_head.

Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever (2018). *Improving Language Understanding by Generative Pre-Training*. Tech. rep. OpenAI.

Radford, A., J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever (2019). *Language Models are Unsupervised Multitask Learners*. Tech. rep. The GPT-2 report; microgpt follows this architecture. OpenAI.

Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems 36 (NeurIPS)*. arXiv:2305.18290.

Raffel, C. et al. (2020). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140. Introduces the C4 web corpus, pp. 1–67.

Robbins, H. and S. Monro (1951). "A Stochastic Approximation Method". In: *The Annals of Mathematical Statistics* 22.3, pp. 400–407.

Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). "Learning Representations by Back-Propagating Errors". In: *Nature* 323, pp. 533–536.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347*.

Sennrich, R., B. Haddow, and A. Birch (2016). "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:1508.07909, pp. 1715–1725.

Shannon, C. E. (1948). "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3, pp. 379–423.

Shannon, C. E. (1951). "Prediction and Entropy of Printed English". In: *Bell System Technical Journal* 30.1, pp. 50–64.

Shao, Z. et al. (2024). "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300*. Introduces GRPO.

Shazeer, N. (2019). "Fast Transformer Decoding: One Write-Head is All You Need". In: *arXiv preprint arXiv:1911.02150*. Multi-Query Attention.

Shazeer, N. (2020). "GLU Variants Improve Transformer". In: *arXiv preprint arXiv:2002.05202*.

Shazeer, N., A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations (ICLR)*. arXiv:1701.06538.

So, D. R., W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le (2021). "Primer: Searching for Efficient Transformers for Language Modeling". In: *arXiv preprint arXiv:2109.08668*. Introduces squared ReLU in the Transformer MLP.

Su, J., Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *arXiv preprint arXiv:2104.09864*.

Sutskever, I., J. Martens, and G. Hinton (2011). "Generating Text with Recurrent Neural Networks". In: *Proceedings of the 28th International Conference on Machine Learning (ICML)*.

Tieleman, T. and G. Hinton (2012). *Lecture 6.5 – RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude*. COURSERA: Neural Networks for Machine Learning.

Touvron, H. et al. (2023). "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971*. Popularised the RMSNorm + SwiGLU + RoPE recipe.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03762.

Xiong, R. et al. (2020). "On Layer Normalization in the Transformer Architecture". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. arXiv:2002.04745; the Pre-LN arrangement used by GPT-2 and microgpt.

Yu, G.-I., J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun (2022). "Orca: A Distributed Serving System for Transformer-Based Generative Models". In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Continuous (iteration-level) batching.

Zhang, B. and R. Sennrich (2019). “Root Mean Square Layer Normalization”. In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. arXiv:1910.07467.